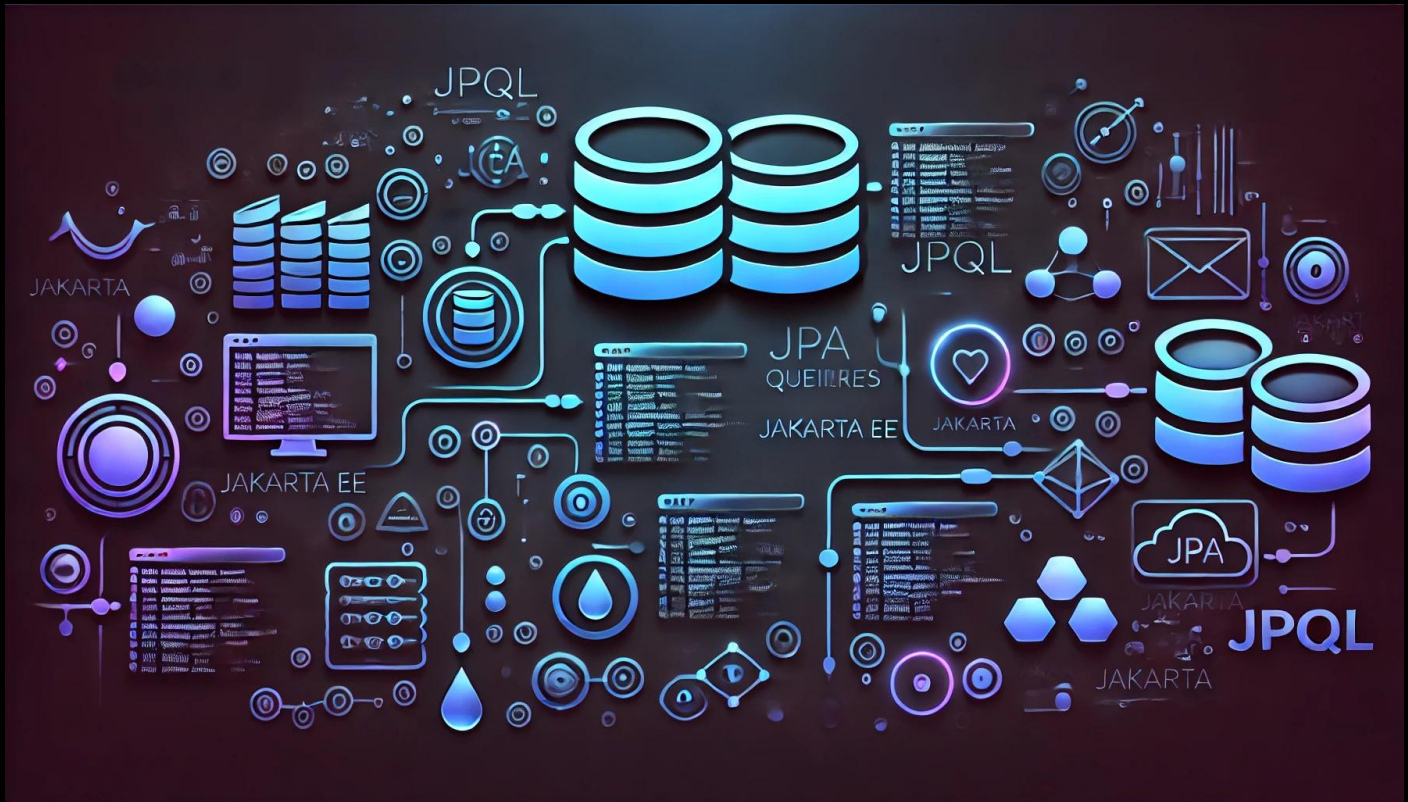


JPQL con JPA y Jakarta EE – parte 3



JPQL con JPA en Jakarta EE – parte 3

En esta guía aprenderemos a realizar **consultas de datos individuales con JPQL (Java Persistence Query Language)** en **JPA**. Estas consultas permiten recuperar **datos específicos de una entidad** en lugar de traer el objeto completo.

Este método es útil cuando solo necesitamos ciertos campos de una entidad y no queremos cargar datos innecesarios en memoria.

✦ Ejemplo práctico:

Queremos obtener solo el **nombre, apellido y email** de todas las personas en la base de datos en lugar de recuperar la entidad completa.

1 Conceptos Básicos

✦ ¿Qué es una tupla en JPQL?

En JPQL, cuando seleccionamos múltiples columnas en una consulta, los resultados se devuelven como **tuplas** (arreglos de objetos).

Ejemplo de SQL:

```
SELECT nombre, apellido, email FROM persona;
```

Equivalente en JPQL:

```
SELECT p.nombre, p.apellido, p.email FROM Persona p;
```

♦ **Diferencia clave:**

- SQL devuelve filas de la base de datos.
- JPQL devuelve **objetos Java (tuplas)**, por lo que es necesario convertirlos (`cast`).

2 Solución Paso a Paso

2.1 Actualizar la Clase `PruebaJPQL`

La clase `PruebaJPQL` ejecutará la consulta y mostrará los resultados.

```
package sga.cliente.jpql;

import java.util.List;
import jakarta.persistence.*;
import java.util.Iterator;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PruebaJPQL {

    private static final Logger log = LoggerFactory.getLogger(PruebaJPQL.class);

    public static void main(String[] args) {
        String jpql = null;
        List<Persona> personas = null;

        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        try (EntityManager em = emf.createEntityManager()) {
```

```
//1. Consulta de todos los objetos de tipo Persona
log.info("\n1. Consulta de todas las Personas");
jpql = "select p from Persona p";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//2. Consulta de una persona por ID
log.info("\n2. Consulta de Persona con ID = 1");
jpql = "SELECT p FROM Persona p WHERE p.idPersona = 1";
Persona persona = em.createQuery(jpql, Persona.class).getSingleResult();
log.info("Persona encontrada: " + persona);

//3. Consulta de una persona por nombre
log.info("\n3. Consulta de Persona por nombre 'Ivonne'");
jpql = "SELECT p FROM Persona p WHERE p.nombre = 'Ivonne'";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//4. Consulta de datos individuales (tupla)
log.info("\n4. Consulta de datos individuales: nombre, apellido, email");

jpql = "SELECT p.nombre, p.apellido, p.email FROM Persona p";
Iterator iter = em.createQuery(jpql).getResultList().iterator();

while (iter.hasNext()) {
    Object[] tupla = (Object[]) iter.next();
    String nombre = (String) tupla[0];
    String apellido = (String) tupla[1];
    String email = (String) tupla[2];

    log.info("Nombre: " + nombre + ", Apellido: " + apellido + ", Email: " + email);
}
}

private static void mostrarPersonas(List<Persona> personas) {
    for (Persona p : personas) {
        log.info("Persona: " + p);
    }
}
}
```

3 Consulta de Datos Individuales en JPQL

3.1 Estructura de la Consulta

✂ **Recuperamos solo 3 columnas** (nombre, apellido, email) de la entidad Persona.

```
String jpql = "SELECT p.nombre, p.apellido, p.email FROM Persona p";
Iterator iter = em.createQuery(jpql).getResultList().iterator();
```

Explicación:

- `SELECT p.nombre, p.apellido, p.email`: Selecciona solo estos campos.
- `createQuery(jpql).getResultList().iterator()`: Ejecuta la consulta y obtiene los resultados como **iterador**.

3.2 Recorrer los Resultados con un Iterador

✂ **Convertimos los resultados en una tupla** (arreglo de objetos).

```
while (iter.hasNext()) {
    Object[] tupla = (Object[]) iter.next();
    String nombre = (String) tupla[0];
    String apellido = (String) tupla[1];
    String email = (String) tupla[2];

    log.info("Nombre: " + nombre + ", Apellido: " + apellido + ", Email: " + email);
}
```

Explicación:

- `Object[] tupla = (Object[]) iter.next();` → Convierte cada fila en un **arreglo de objetos**.
- `tupla[0], tupla[1], tupla[2]` → Recupera cada campo por su **posición en la consulta**.
- **Se usa `log.info()` para imprimir los resultados en consola.**

Resultado Esperado

Si ejecutamos la clase `PruebaJPQL`, veremos en la consola:

4. Consulta de datos individuales: nombre, apellido, email

```
[EL Fine]: sql: ServerSession(1081769770)--Connection(854653864)--SELECT NOMBRE, APELLIDO, EMAIL
FROM PERSONA
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - Nombre: Rafael, Apellido: Juarez, Email: rjuarez@mail.com
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - Nombre: Ivonne, Apellido: Lara, Email: ilara@mail.com
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - Nombre: Maria, Apellido: Esparza, Email: mesparza@mail.com
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - Nombre: Hugo, Apellido: Hernandez, Email: hhernandez@mail.com
```

Esto confirma que **las consultas JPQL** están funcionando correctamente 🎯.

5 Conclusión

🎯 En esta guía aprendimos a realizar **consultas de datos individuales con JPQL en JPA**.

♦ Puntos clave:

- ✓ **JPQL** permite seleccionar datos específicos en lugar de objetos completos.
- ✓ Los resultados se devuelven en **tuplas (arreglos de objetos)**.

🚀 ¡Ahora puedes aplicar esta técnica en tus propios proyectos!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)